Design Document

Uplift: AI-Powered Student Success Assistant (Education domain, OULAD)

Version 6.0.0. Figures quoted here are read from the evaluation artifacts when this document is built (`scripts/build_docs.py`); the full results are in `docs/Evaluation_Report.pdf`.

Contents. 1 Problem statement · 2 Solution overview · 3 Architecture design · 4 Technology selection · 5 Retrieval strategy · 6 Agent design · 7 Prediction models · 8 Guardrails and responsible AI · 9 Evaluation strategy · 10 Feedback loop · 11 Challenges and trade-offs · 12 Limitations and future work · A PRD traceability · B Repository layout

1. Problem statement

Universities need to find students who are struggling academically, disengaging from online learning or likely to withdraw, early enough to help. Module teams have the data (student profiles, assessment results, online activity, registration history, past outcomes), but reading thousands of records by hand is slow and inconsistent, and generic dashboards answer "who is at risk" without saying why, what happened to similar students before, or what to do.

The brief asks three questions, and the system answers each one with evidence:

Question in the brief	How Uplift answers it
Has a student with similar academic and activity patterns struggled or withdrawn before?	Hybrid RAG over every finished historical enrolment (earlier presentations only): the closest cases, what happened to them after this point, and the outcome mix of the 50 nearest.
What factors are associated with poor performance or withdrawal?	Per-student drivers (TreeSHAP) on a calibrated model; cohort-level factor analysis, engagement trends and assessment analytics from complete records.
Which students should be prioritised for support?	A ranking learned for limited advisor capacity, with reasons per student, and a multi-agent workflow that drafts a supportive plan and message for the advisor to review.

Users. Tutors, student-support advisors and module leads of an Open University module. **Business objective.** Contact the right students earlier with a better first message, so more of them finish: fewer withdrawals and fails per advisor hour, decisions an advisor can explain, and no automated adverse decisions.

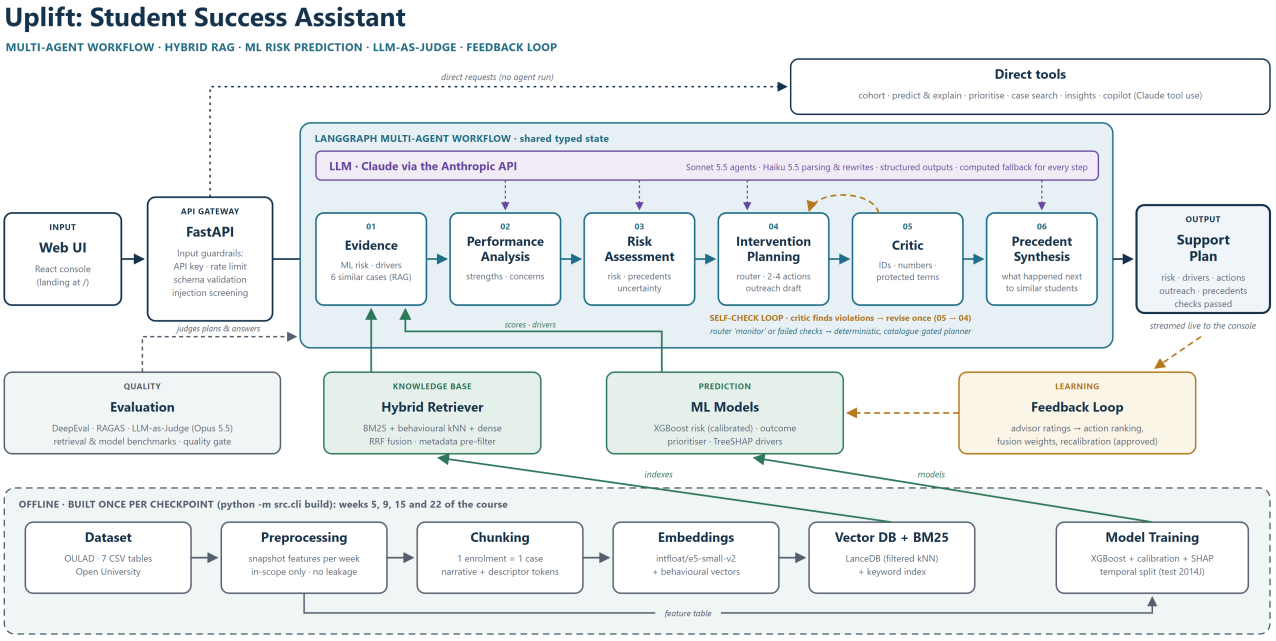
2. Solution overview

Uplift is a microservice (FastAPI) with a React console. It works "as of" a point in the course: models, indexes and rankings are built at 4 checkpoints (weeks 5, 9, 15, 22), and only students still enrolled at that point are scored.

- **Early warning.** Calibrated withdrawal risk with tiers and drivers, plus a four-class outcome model.
- **Who first.** A prioritised contact list for a chosen advisor capacity, with the reasons for each student.
- **Similar students (RAG).** Hybrid search (behavioural vectors + BM25 + dense embeddings in LanceDB) over historical cases, by student or by plain-English description, with metadata filters (course, region, education, age band, credits, previous attempts, outcome).
- **Support plan (multi-agent).** A LangGraph workflow: performance analysis, risk assessment, routing, intervention planning, a critic that checks every draft in code, one revision, and a deterministic fallback; then a summary of what happened to similar students.
- **Copilot.** Questions in plain English answered through tool calls over the same computed functions, streamed, with every number checked.
- **Insights.** Factors, engagement trends by outcome, assessment analytics, fairness audit.
- **Evaluation and learning.** DeepEval, RAGAS, LLM-as-judge, human review, a usage and cost ledger, and an approval-gated feedback loop.

Design principles: the LLM narrates and selects, code decides facts; every number says what it is and as of when; generated text is labelled and checked; everything still works with the LLM switched off.

3. Architecture design



3.1 How to read the diagram

Offline (bottom), built once per checkpoint by `python -m src.cli build`:

1. **Dataset**: the seven OULAD CSV tables.
2. **Preprocessing** (`src/pipeline`): snapshot features at the checkpoint day, students still enrolled only, leakage controls; writes the feature table.
3. **Chunking** (`src/rag/descriptors.py`): one document per finished enrolment: discretised descriptor tokens and a short narrative (numbers are banded, never embedded as text).
4. **Embeddings**: narratives encoded with intfloat/e5-small-v2 ("passage: " prefix); snapshot features become behavioural vectors weighted by the model's attributions.

5. **Vector database + BM25** (`src/rag/vector_store.py`, `hybrid.py`): both vector channels in LanceDB with the filterable metadata as columns; a BM25 index over the descriptor tokens.
6. **Model training** (`src/models`): XGBoost withdrawal and outcome models, calibration, tier thresholds, TreeSHAP, the prioritiser; temporal split.

Online (middle), per request: the **Web UI** (React console) calls the **FastAPI** gateway (API key, rate limit, schema validation, injection screening). Direct tools (cohort, prediction, ranking, case search, insights, copilot) are answered from the computed services. A support-plan request runs the **LangGraph workflow**: (01) evidence assembly from the **ML models** and the **hybrid retriever**, (02) performance analysis, (03) risk assessment, (04) routing and intervention planning, (05) the critic with one revision, (06) precedent synthesis; the **LLM** (Claude) drafts text in 02, 03, 04 and 06, each with a computed fallback. The **support plan** streams back to the console step by step. **Evaluation** scores agent outputs offline and on a sample of live traffic; the **feedback loop** turns advisor ratings into approved updates.

3.2 Components

Component	Code	Responsibility
Entry point	`src/main.py`	`uvicorn src.main:app`: API under `/api`, console at `/`
API layer	`src/api/`	Routes, schemas and input validation, security (API key, admin key, rate limit, body size), jobs, server-sent events, route documentation
Service	`src/services/student_success.py`	One model bundle per checkpoint; cohort, student, ranking, similar cases, analysis, insights
Data pipeline	`src/pipeline/`	OULAD to snapshot features, scoping, leakage controls, weekly activity grid
Models	`src/models/`	Withdrawal risk, outcome, prioritiser, temporal split, feature contract
RAG	`src/rag/`	Descriptors and narratives, hybrid retriever, vector store, retrieval benchmarks
Agents	`src/agents/`	LLM client (Claude), workflow and LangGraph graph, guardrails, copilot, briefing / synthesis / outreach / query understanding
Evaluation	`src/evaluation/`	Offline AI evaluation, LLM judge, DeepEval suite, model analysis and fairness, model and embedding comparisons
Feedback	`src/feedback/`	Feedback store and the approval-gated improvement loop
Utilities	`src/utils/`	Settings (`config.py`), I/O and logging helpers
Reporting	`src/reporting/`	Model card from artifacts
CLI	`src/cli.py`	`prepare, train, index, evaluate, llm-eval, quality-gate, report, build`, benchmarks
Console	`frontend/`	React 19 + TypeScript (Vite): course clock, students, case finder, copilot, insights, model quality, feedback

3.3 Data flow of one request: "draft a support plan"

1. The console posts `{student, checkpoint}` to `POST /api/jobs/analyze`; the API validates it (pydantic schema, module vocabulary, positive IDs) and returns a job id (202). The console polls `GET /api/jobs/{id}`.
2. The service resolves the student at the checkpoint (409 if they had already left), scores withdrawal risk and outcome, and explains the risk with TreeSHAP drivers.
3. The retriever finds similar students from earlier presentations of the same module (behavioural + lexical channels, RRF), returns 6 cases as stories and the outcome mix of the 50 nearest.
4. Facts become numbered observations (`OBS_...`) and the eligible actions (`ACT_...`) are filtered from a catalogue by rules on the evidence. Sensitive attributes are not part of the evidence.
5. LangGraph runs the agents on a shared typed state; every LLM output is validated (IDs, numbers, terms, length) and replaced by the computed version if it fails twice.
6. Precedent synthesis summarises what happened to the similar students (numbers checked against the retrieved cases).
7. The result is stored with a recommendation id; a sampled share is judged online; the advisor can edit the message, rewrite it for SMS or another language, ask the judge, and leave feedback.

3.4 Deployment (local)

One process: `uvicorn src.main:app --reload` serves the REST API and the built console (`frontend/dist`), so a reviewer needs Python only. Models and indexes load lazily per checkpoint. Background jobs and rate limits are in memory, or in SQLite (`SHARED_STATE=sqlite`) for several workers on one host. Docker files are included but optional.

4. Technology selection

Layer	Choice	Alternatives considered	Why this choice (evidence)
LLM	**Claude** via the Anthropic API, three tiers: claude-haiku-5-5 (query understanding, outreach rewrites, note themes), claude-sonnet-5-5 (agents, copilot, precedent synthesis, briefing), claude-opus-5-5 (judge only)	one model for everything; judge equal to generator	Measured on the same prompts and checks (`llm-compare`): for query parsing the fast tier reached filter F1 0.890 at 0.0003 USD per call vs 0.877 for the main tier; for precedent synthesis the main tier had the highest judge pass rate (100% vs 38%). A different, stronger judge avoids self-grading. Structured outputs, tool use, prompt caching and streaming are used.
Embedding model	**intfloat/e5-small-v2** (384 dims, CPU)	all-MiniLM-L6-v2, all-mpnet-base-v2, bge-small-en-v1.5, gte-small	Compared on the same 25k narratives (`embedding-eval`): best precedent quality and near-best typed search at the same query latency as MiniLM; mpnet is 2x slower and not better. Query / passage prefixes are recorded in the index manifest.

Layer	Choice	Alternatives considered	Why this choice (evidence)
Vector database	**LanceDB** (embedded)	Chroma, FAISS, Qdrant server, NumPy arrays	No extra service to run; metadata filters are pushed down as a pre-filter so they never drop true neighbours; exact kNN at ~25k vectors per checkpoint (identical top-10 to a NumPy scan on 1,800 sampled searches); can add ANN indexes if the corpus grows.
Agent framework	**LangGraph** state graph	plain Python orchestration (kept as reference), LangChain agents, LlamaIndex, CrewAI	Explicit, renderable control flow with conditional edges (router, critic, revision); our nodes keep their own guardrails, cache and cost ledger. A parity test proves LangGraph and the reference orchestrator give identical results.
Predictive models	**XGBoost** + calibration + **TreeSHAP**	logistic regression (kept as baseline), random forest, neural nets	Best ranking on tabular features, exact per-student explanations, CPU-fast; ROC-AUC 0.726 vs 0.703 for the logistic baseline.
API	**FastAPI** + Uvicorn, pydantic v2	Flask, Django	Typed request validation, automatic OpenAPI / Swagger, async streaming (SSE), background jobs.
User interface	**React 19 + TypeScript** (Vite), TanStack Query / Table, Recharts, Motion	Streamlit (the guideline's default)	The brief asks for a simple front end; Streamlit would cover it. React was chosen to go further: real URLs per student, virtualised tables of ~2,000 students, streamed copilot answers with tool-call cards, keyboard and screen-reader support (Lighthouse accessibility 100). It is served by the same FastAPI process, so it needs no Node.js at demo time.
Evaluation	**DeepEval**, **RAGAS**, own LLM-judge, pytest	a single metric	Each covers a different failure: DeepEval (faithfulness, relevancy, custom G-Eval criteria), RAGAS (groundedness of retrieval-augmented text), judge with order-swapped A/B (quality vs the computed plan), pytest (behaviour without any LLM). RAGAS runs in its own virtual environment because its LangChain dependencies conflict with the pinned service.
Data / ML stack	pandas, NumPy, scikit-learn, SciPy	Spark	10.6M activity rows fit in under 2 GB with compact types; a full build of all checkpoints takes minutes on a laptop.

5. Retrieval strategy

5.1 Knowledge base and chunking

The knowledge base is every **finished, in-scope historical enrolment**: 25,204 documents at week 15. **One document per enrolment, no further chunking.** A case is a short, self-contained record; splitting it would separate the signals from the outcome they led to. Each document holds:

- the snapshot features (numeric) as of the same checkpoint, so a case is compared at the same point in its course;
- **descriptor tokens**: features banded relative to the cohort (`score_low`, `missed_most`, `engagement_declining`, `long_inactive_streak`, `credits_heavy`, `repeat_attempt`...). Numbers never reach the text channels: "45" and "46" share no token, so raw numbers made the original lexical and dense search no better than random;
- a ~60-word **narrative** written from the descriptors for the dense channel;
- **metadata** columns for filtering (module, presentation, region, education, age band, credits, previous attempts, outcome) and the later trajectory used to tell "what happened next".

5.2 Embedding approach

- **Dense**: narratives encoded with intfloat/e5-small-v2 using its "passage: " prefix; queries use "query: ". The model and prefixes are stored in the index manifest so queries always match the stored vectors. On rebuilds, vectors of unchanged narratives are reused.
- **Behavioural**: robust-scaled snapshot features weighted by the withdrawal model's mean absolute attribution per feature, so "similar" means similar in the ways that matter for outcomes (supervised similarity).

5.3 Retrieval method

Three channels rank the eligible cases: **behavioural kNN** (weighted Euclidean, exact), **BM25** over descriptor tokens, and **dense** cosine similarity. Both vector channels run in LanceDB with the metadata filter applied as a pre-filter and exact search (so filters never remove true neighbours). Guards: only presentations that finished **before** the query's presentation are searchable (the current cohort's outcomes are unknown), the student's own enrolments are excluded, and only students still enrolled at the checkpoint are indexed.

5.4 Hybrid search

Channel results are fused with **weighted Reciprocal Rank Fusion** (k = 60, top 200 per channel). RRF needs no score calibration across channels. Weights are tuned on the validation presentation: for student-to-student search the behavioural channel carries most weight (profiles are numeric); for typed search the weights are tuned per query parser. Typed requests are first **understood**: "repeat students in FFF with heavy workloads who dropped out" becomes filters (`code_module=FFF`, `num_of_prev_attempts>=1`, `final_result=Withdrawn`) plus behaviour descriptors (`credits_heavy`). The fast Claude tier does this with structured output; a rule parser is the fallback, and explicit filters set in the console always win.

Results at week 15: neighbour-vote AUC (do the retrieved students' real outcomes predict this student's?) 0.835 for hybrid vs 0.512 for random; typed search precision@10 0.862 with the AI parser.

5.5 How retrieved context is used (RAG)

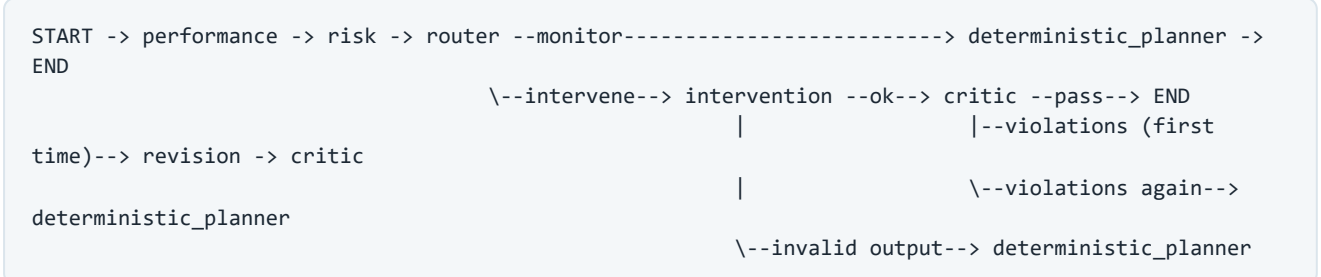
- The **6 closest cases** go to the agents and the precedent synthesis as short stories ("withdrew in week 25, activity fell 87% after week 13"). Few enough to keep prompts readable and cheap.
- The **outcome mix of the 50 nearest** (with a 95% interval) is the statistical answer to "did students like this struggle?"; more neighbours give steadier estimates. Both numbers come from a sensitivity study (Evaluation Report, model card section "Retrieval strategy").
- The synthesis may only use numbers present in the retrieved cases; the code check rejects anything else.

6. Agent design

6.1 Agent roles

Agent (graph node)	Role	LLM	Fallback
Evidence assembly	Scores, drivers, similar cases, numbered observations, eligible actions	no	(is computed)
Performance analysis	Strengths and concerns, citing observation IDs	claude-sonnet-5-5	observations verbatim
Risk assessment	Calibrated risk, drivers, precedents, uncertainty, in plain words	claude-sonnet-5-5	templated from the model outputs
Router	Intervene or monitor (tier, P(fail or withdraw), number of concerns)	no	rule in code
Intervention planning	2 to 4 actions from the eligible catalogue (11 actions), outreach message, rationale citing IDs	claude-sonnet-5-5	deterministic, catalogue-gated planner
Critic	Checks IDs, eligibility, message length, protected / punitive / clinical / stigma terms, numbers	no	requests one revision, then fallback
Revision	The intervention agent again, given the critic's findings	claude-sonnet-5-5	deterministic planner
Precedent synthesis	What happened to similar students and what it implies	claude-sonnet-5-5	computed summary
Judge (evaluation)	Six-criterion score of the plan; verdict computed in code	claude-opus-5-5	not run

6.2 Workflow



Implemented as a LangGraph `StateGraph` (`src/agents/graph.py`); `WORKFLOW_ENGINE=python` runs the reference orchestrator with identical results (`tests/test_graph.py`). The console shows each step as it completes.

6.3 Agent communication

Agents do not chat with each other. They read and write one **typed shared state**: the evidence (observations with IDs, eligible actions, model outputs, retrieved cases), each agent's structured output, the critic's violations and a trace. LLM agents receive only the evidence (never raw records or sensitive attributes) and must answer in a JSON schema (Claude structured outputs) that cites observation and action IDs. That is what makes their claims checkable: an unknown ID, an ineligible action or a number not in the evidence is a violation.

6.4 Copilot

A tool-using assistant over 8 read-only tools (student, timeline, similar students, cohort ranking, cohort overview, risk factors, assessment insights, model performance). Claude decides which tools to call (in parallel where possible); the answer streams to the console with the tool calls shown; every number in the answer is checked against the tool results and, if needed, the answer is revised. Injection attempts and off-topic or sensitive requests are refused. With the LLM off, a rule-based router answers common questions with the same tools.

6.5 Other LLM features

Weekly **cohort briefing**, **outreach rewriting** (email, SMS, phone script; tone; five languages with an English back-translation), **query understanding** for case search and **themes in advisor notes**. Each has a schema, code checks and a computed fallback, and an offline evaluation.

7. Prediction models

Model	Target	Method	Test result (week 15)
Withdrawal risk	withdraws after the checkpoint	XGBoost, Platt calibration, tiers from validation (Medium catches 80% of withdrawals, High maximises F1 above Medium)	ROC-AUC 0.726, PR-AUC 0.263 (base 0.113), ECE 0.009
Final outcome	Distinction / Pass / Fail / Withdrawn	XGBoost multiclass, class weighting chosen on validation, temperature scaling	macro-F1 0.485 vs 0.460 logistic; P(fail or withdraw) ROC-AUC 0.852
Prioritiser	fail or withdraw, at advisor capacity	weighted blend of six signals, exhaustive simplex grid, one-standard-error rule	precision at 10% 98% (random 34%)

Explanations are exact TreeSHAP attributions, shown as drivers that raise or lower risk relative to a typical student. Separate models at each checkpoint let accuracy and base rates change through the course: ROC-AUC rises from 0.686 at week 5 to 0.773 at week 22, while fewer students remain to help.

8. Guardrails and responsible AI

Input guardrails (API). API key on every route, separate admin key for applying changes, per-client rate limit, request size limit, strict pydantic schemas (module and presentation vocabularies, positive IDs, ranges, what-if values checked against the feature contract), text sanitisation and prompt-injection screening for free text, generic error bodies with a request id.

Output guardrails (every LLM output). Cited IDs must exist; numbers must appear in the evidence (with tolerance); actions must be eligible; no protected attributes, punitive, clinical or stigmatising language; channel length limits; structured schema. Failing output is revised once or replaced by the computed version, and the console labels what was drafted by AI and what was computed.

Sensitive attributes. Gender, region, deprivation band, age band, prior education and disability are never model inputs, never drivers and never sent to the LLM; including them would change ROC-AUC by only 0.002. They are used for the fairness audit and as retrieval filters only.

Fairness audit. Equal-opportunity gaps of the High tier are tested against the gap random labels would give at the same group sizes (permutation null, overall and within module), with proxies and a per-group threshold review shown for the equity office (never applied automatically). The audit is in the Evaluation Report.

Human in the loop. Uplift recommends; an advisor decides and edits every message. It is not for grading, admissions or sanctions.

9. Evaluation strategy

What	How	Metrics	Where
Prediction models	temporal split, in-scope students, once on the test presentation; baselines; bootstrap CIs	ROC-AUC, PR-AUC, Brier, ECE, precision / recall at capacity, macro-F1	`src/models`, model card
Retrieval	600 test students vs earlier cohorts (outcome-based relevance, not circular); 40 labelled typed queries; top-K, pool and RRF sensitivity; five embedding models	neighbour-vote AUC, outcome agreement@10, nDCG@10, precision@10, filter / behaviour match	`src/rag/benchmark.py`, `src/evaluation/embedding_eval.py`
Agents and generated text	code checks; LLM-as-judge (claude-opus-5-5) with anchored rubric and computed verdict; order-swapped A/B vs computed plan	guardrail pass rate, judge pass rate and criterion scores, A/B win rate and position consistency	`src/evaluation/offline_eval.py`, `judge.py`

What	How	Metrics	Where
DeepEval	faithfulness, answer relevancy, three G-Eval criteria (actionable, supportive and non-punitive, no protected-attribute reasoning)	mean, 95% CI, pass rate	`src/evaluation/deepeval_suite.py`
RAGAS	precedent synthesis and copilot answers with their retrieved context	faithfulness (hallucination rate), response groundedness, context relevance	`scripts/ragas_eval.py`
Copilot	golden questions with known answers; adversarial prompts	correct answer, right tool, grounded and safe; safe rate	`src/evaluation/offline_eval.py`
Query understanding	60 templated + 6 paraphrased labelled queries	filter precision / recall / F1, exact match	`src/evaluation/search_queries.py`
Software	pytest suite on a synthetic fixture: no network, no LLM	pass / fail	`tests/`
Online	sampled live judging (`ONLINE_EVAL_SAMPLE_RATE`), human review tab, usage and cost ledger	judge verdicts, Cohen's kappa, cost per feature	`/api/evaluations/*`, `/api/llm/usage`
Release	quality gate turns all of the above into pass / fail checks	22 checks + 3 fairness warnings	`python -m src.cli quality-gate`

10. Feedback loop

Advisors rate each plan (usefulness, will act on it, agree with the risk), keep or drop each action, mark useful precedents and record real outcomes. Learners turn this into proposals with minimum-sample guards: re-rank the action catalogue by acceptance, re-weight retrieval channels from precedent usefulness, recalibrate risk on recorded outcomes, and monitor tier drift. Nothing changes until an admin applies a proposal (`POST /api/feedback-loop/run` with `apply=true`); applied changes are versioned with approver and time, and the offline evaluation is re-run.

11. Challenges and trade-offs

Challenge	Decision	Trade-off / limitation
Leakage: an earlier version reported ROC-AUC 0.90 by scoring students who had already left	Score only students still enrolled; temporal split; `date_unregistration` for scope only	Honest ROC-AUC is 0.726: lower, but it is what an advisor would actually get

Challenge	Decision	Trade-off / limitation
Early vs accurate	Four checkpoints with separate models	Earliest week is least accurate (0.686) but has the most students left to help
Numbers in text retrieval	Band numbers into descriptor tokens; numeric similarity in the behavioural channel	Banding loses fine detail; the behavioural channel recovers it
Dense channel on profile search	Kept for typed search; tuner gives it little weight for student-to-student search	Narratives are templated, so dense acts partly as a soft lexical channel
LLM reliability	LLM narrates and selects; code decides facts; checks + fallback everywhere	More engineering; some AI drafts are replaced by computed text (8% of syntheses)
Cost and latency	Three Claude tiers by measured need; response cache; prompt caching; budget cap; background jobs and streaming	The cache must be invalidated when prompts change (prompt version in the key)
Self-grading bias	Judge is a different, stronger model; verdict computed from scores; order-swapped A/B; human review	Judge cost; human labels still needed to calibrate trust
Dependency conflicts	RAGAS in its own environment	Two environments to maintain
Fairness	Sensitive attributes out of the model; audit with permutation null; threshold review for humans	A disability gap remains and is reported, not hidden; equalising it is a policy decision
Over-tuning	A promising feature set improved validation but not the untouched test set, so it was rejected	Leaves possible gains unadopted rather than tuning on the test set
UI scope	React instead of Streamlit	More code than Streamlit; served by the API so the demo still needs one command

12. Limitations and future work

- Associations, not causes: drivers explain the model, not the student.
- One institution, 2013 to 2014, one test presentation; revalidate before use elsewhere and monitor drift (feedback loop).
- No real advisor feedback or human labels yet; the loop and the human-review tab are ready for them.
- Future: approximate-nearest-neighbour indexes for larger corpora, a job queue and gateway rate limits for multi-host deployment, multilingual narratives.

Appendix A. PRD traceability

PRD requirement	Implementation	Evidence
Basic RAG for historical student similarity search	Case index of in-scope enrolments in LanceDB; retrieved cases ground the risk agent, plan and precedent synthesis	`src/rag/`, `POST /api/retrieve-similar`, Evaluation Report 3
Hybrid search (keyword + semantic)	BM25 + behavioural kNN + dense (e5-small-v2), weighted RRF	`src/rag/hybrid.py`, Evaluation Report 3.1-3.2
Student performance and risk classification agent	Performance and risk agents (LangGraph) on calibrated model outputs, tiers High / Medium / Low	`src/agents/workflow.py`, `graph.py`
Metadata filtering (course, region, education, age band, credits, previous attempts)	Filters pushed down to LanceDB as a pre-filter; also understood from plain English	`src/rag/vector_store.py`, case finder
Basic student-support recommendations	Intervention agent over an eligible action catalogue + outreach draft	`POST /api/analyze`
Input validation guardrails	pydantic schemas, vocabularies, ranges, size limit, injection screening, rate limit, API key	`src/api/schemas.py`, `security.py`, `guardrails.py`
Student prioritisation	Learned ranking for advisor capacity with reasons	`POST /api/prioritize`
Core functionality through API endpoints	39 documented operations	`docs/API_Documentation.pdf`, `/api/docs`
DeepEval	Faithfulness, relevancy, three G-Eval criteria	Evaluation Report 4
Performance and withdrawal-risk prediction models	XGBoost withdrawal and outcome models, four checkpoints	Evaluation Report 10
LLM-as-Judge	claude-opus-5-5, six criteria, order-swapped A/B, online sampling	Evaluation Report 6
Multi-agent workflow (performance, risk, intervention)	LangGraph graph with router, critic and revision	Section 6
Engagement and performance trend analysis	Weekly trends by outcome, divergence weeks, student trends	`GET /api/trends/*`, Insights
Explainable analysis of student, assessment and activity factors	TreeSHAP drivers, factor quintiles, assessment AUCs	`GET /api/insights/*`
Feedback loop for continuous improvement	Feedback store, learners with guards, admin-approved apply	Section 10
Simple front-end	React console (served by FastAPI)	`frontend/`
Deliverables	Architecture diagram, design document, code, presentation	`architecture/`, `design/`, `src/`, `presentation/`

Appendix B. Repository layout

```
architecture/     Architecture_Diagram.pdf / .png / .jpeg / .svg
design/           Design_Document.pdf (this document) and its Markdown
src/              api/, agents/, rag/, services/, utils/, models/, pipeline/, evaluation/, feedback/,
reporting/, cli.py, main.py
frontend/         React console (source and built dist/)
data/             sample_data/ (committed OULAD sample); raw/ and processed/ are local only
docs/             API_Documentation, Evaluation_Report, Dataset_Details, Presentation_Guide (PDF +
Markdown), model card,
                  decisions, security, deployment, agent graph
tests/            pytest suite
presentation/     Project_Presentation.pptx (+ PDF)
scripts/          documents, diagram, sample data, RAGAS, API sample capture
```